



CASE VÍSSIMO – ANALISTA DE DADOS

Objetivo

Construir um pipeline completo de dados, desde a ingestão até a visualização, utilizando ferramentas modernas e práticas atuais de engenharia e análise de dados.

O candidato deverá demonstrar capacidade de:

- Consumir APIs
- Estruturar dados
- Versionar código
- Utilizar Python
- Utilizar banco de dados para armazenamento
- Aplicar IA para acelerar desenvolvimento
- Construir dashboards conectados diretamente à fonte de dados

Desafio

Construir uma solução que permita analisar e comparar dados meteorológicos entre diferentes regiões e períodos.

1. Ingestão de Dados (API)

- Consumir dados da API pública da Open-Meteo
- Definir quais variáveis climáticas serão utilizadas (temperatura, chuva, vento, etc.)
- Implementar coleta automatizada via Python

2. Armazenamento e Modelagem

- Criar estrutura de banco relacional (PostgreSQL, SQLite, DuckDB ou similar)
- Caso prefira um banco hospedado na nuvem, recomendamos a plataforma Supabase (Gratuita com limite de 500mb para o banco de dados)
- Definir modelagem (fato + dimensões (Dimensional) OU estrutura simples (Relacional) bem organizada)
- Persistir os dados coletados

3. Pipeline de Dados

- Organizar o fluxo: coleta → transformação → carga

4. Dashboard (Sem uso de ferramentas como Looker ou Powerbi)

Construir um dashboard web com:

- Frontend (React, HTML + JS ou similar)
- Backend simples (Node.js, Python Flask ou similar)

O dashboard deve:

- Conectar diretamente ao banco ou API
- Permitir filtros (região, período, variável climática)
- Exibir visualizações (linha, barra, comparativos)

5. Uso de IA (obrigatório)

O candidato deve:

- Utilizar IA (ChatGPT, Gemini, etc.) no desenvolvimento
- Explicar claramente onde e como usou IA:
 - geração de código
 - otimização
 - criação do dashboard

6. Versionamento

- Repositório no GitHub obrigatório
- Código organizado e com commits claros

Diferenciais (não obrigatórios)

- Deploy do dashboard (Vercel, Render, etc.)
- Uso de Docker
- Estrutura de projeto profissional (separação front/back)
- Tratamento de erros e logs
- Uso de variáveis de ambiente (.env)
- Pequena camada de API própria (ex: /weather)

Entregáveis

- Repositório GitHub com:
 - Código Python (ingestão)
 - Código do dashboard (front + back)
 - Script SQL (modelagem)
- README explicando:
 - Arquitetura da solução
 - Como rodar o projeto
 - Onde utilizou IA e por quê

Links de apoio

- Open-Meteo: <https://open-meteo.com/en/docs>
- (Opcional) Numbeo: <https://www.numbeo.com/cost-of-living/>